

Lava API Android App — Design Spec

- **Date:** 2026-06-02
- **Status:** Approved (operator approved all sub-projects; subagent-driven execution authorized)
- **Author:** Claude (Opus 4.8) under operator direction
- **Classification:** project-specific (the on-device-API app + Go SQLite backend are Lava-specific; the additive-backend-selector pattern is universal and noted as such inline)
- **Origin:** Operator directive 2026-06-02 — "We MUST HAVE as a separate installable application ... the API service ... bootup the whole API on an Android device locally and expose it to the whole network ... User should be able to stop/restart/track/start ... proper System notification ... extend client app (onboarding + settings) ... download-or-launch flow ... API appears in the list of available API instances ... other users on the same network discover & use it ... in-depth docs/diagrams ... maximal test coverage with real evidence, zero bluffs ... distribute all three apps (API, Client, new API Android) debug+release via Firebase, same signing key."

0. Operator decisions (locked)

1. **Runtime:** Extend `lava-api-go` to support a **SQLite** storage backend as an *additive, config-selectable* option. Postgres remains the default and is **not modified in behavior**. The new Android app **wraps the cross-compiled Go server** and uses SQLite. "Nothing already working can be broken — only extended for more flexibility."
2. **Go embedding: In-process via gomobile bind** (.aar), not a child process. Still a cross-compiled Go artifact; cleaner Start/Stop/Status lifecycle; real-HTTP instrumented testing.
3. **Discovery identity:** New, **distinct** on-device identity in the client's instances list, while the existing host Go/Postgres instance keeps working unchanged.
4. **Scope:** Decomposed into sequenced sub-projects; build the API app first. All sub-projects approved.

5. **Distribution:** Full-auto build → test → distribute, with a **hard §6.Z refusal** if Challenge tests do not execute green against the exact artifact, and any missing secret/device surfaced as a hard blocker (no bluff).

1. Sub-project decomposition (all approved)

#	Sub-project	This spec
1	Lava API Android app + the additive <code>lava-api-go</code> SQLite/Android extension it depends on	Detailed below
2	Client integration — onboarding API step + Settings "Run API on this device" with install-or-launch flow; clear non-confusing copy	Roadmap §8; own spec later
3	Docs/diagrams/guides — architecture, on-device API flow, mDNS, user manual; extend all existing docs	Roadmap §8; own spec later
4	Build + full test + dual-variant Firebase distribution of all 3 apps (API host service artifacts, Client Android, new Lava API Android), same keystore, via Firebase CLI, §6.Z gate enforced	Roadmap §8; own spec later

Each sub-project gets its own spec → plan → impl cycle. This document fully specifies #1 and records #2–#4 as committed roadmap so the sequencing is unambiguous.

2. `lava-api-go` additive SQLite backend (the only existing code touched)

2.1 Principle (universal pattern)

Introduce a **backend selector** whose default reproduces today's behavior byte-for-byte. The current code becomes "the postgres branch," never "the only path." Every existing deployment that sets `LAVA_API_PG_URL` and nothing else behaves identically.

2.2 Config (additive)

- `LAVA_API_STORAGE_BACKEND` ∈ {postgres (default), sqlite}.
- `LAVA_API_SQLITE_PATH` (required iff backend=sqlite).

- LAVA_API_PG_URL required **iff** backend=postgres (today's hard-dep, now conditional on the default branch — unchanged for existing users).
- No env var renamed or removed. New vars carry placeholders in `.env.example` (§6.R).

2.3 Storage boundary

- Define a Go storage interface over what the internal/cache (and any other Postgres-bound) layer needs.
- Two implementations:
 - **postgres** — wraps the existing `submodules/cache/pkg/postgres` adapter; behavior unchanged.
 - **sqlite** — backed by modernnc.org/sqlite (pure-Go, no CGo → cross-compile cleanly for Android arm64/arm/amd64; already a transitive dep from the LVA tickets work). Migrations adapted for SQLite dialect under `migrations/sqlite/`.
- Selection happens once at startup in the composition root; handlers/middleware are backend-agnostic.

2.4 internal/mobile package (gomobile surface)

- `Start(configJSON string) error` — parse config, init SQLite storage, start the existing server on the configured bind/port, return when listening (or error).
- `Stop() error` — graceful shutdown of the running server.
- `Status() string` — JSON: state, bind addr, port, request count, storage backend, version.
- Designed for `gomobile bind` (exported types kept to strings/ints/errors for a clean JNI surface).

2.5 Regression & parity guards (§6.A / §6.J)

- Existing Postgres unit/contract/parity/e2e tests remain **untouched and green**.
- New `*_sqlite_test.go` mirror the Postgres suites and assert **identical JSON contract** (a parity test feeds the same requests to both backends and asserts equal response shape + status + key fields).
- `internal/mobile` test: call `Start` against a loopback port, issue a **real HTTP request**, assert real JSON body, `Stop`, assert port closed. Falsifiability rehearsal recorded in commit body.

3. Android app — module & component architecture

3.1 New Gradle modules

- **:api-app** — applies `lava.android.application`; `applicationId = digital.vasic.lava.api`, `debug applicationIdSuffix = .dev` → `digital.vasic.lava.api.dev`; `versionCode/versionName` independent of the client, tracked in `.lava-ci-evidence/distribute-changelog/`. Reuses `core:designsystem`, `core:notifications`, `Compose`, `Hilt`, `Orbit`.
- **:core:apiengine** — thin Kotlin wrapper over the generated `gomobile .aar:ApiEngine` interface (`start(config): Result`, `stop()`, `status(): ApiStatus`), a real impl delegating to the `.aar`, and a `FakeApiEngine` for unit tests that is **behaviorally equivalent** (enforces start-before-status, error propagation — §Third Law).
- The `.aar` is produced by a build step (`gomobile bind`) wired into the module; checked-in vs built-on-demand decided in the plan (prefer reproducible local build per Local-Only CI/CD; cache the artifact).

3.2 ApiEngineService (foreground Service)

- Owns the Go server lifecycle: **Start / Stop / Restart**.
- Holds `WifiManager.MulticastLock` (mDNS), `WifiManager.WifiLock`, and a partial `WakeLock` so the server survives doze while running; all released on Stop.
- Registers/unregisters the mDNS service via `NsdManager` (see §4).
- Emits/updates the foreground notification (see §5).
- Survives configuration changes; exposes state via a `StateFlow` the UI observes.

3.3 Landing UI (MVI / Orbit)

- `ApiControlViewModel` : `ContainerHost<ApiControlState, ApiControlSideEffect>` + sealed `ApiControlAction`.
- State machine: `Stopped` → `Starting` → `Running(url, ips, requestCount)` → `Stopping` → `Stopped`, plus `Error(message)`.
- Screen shows: big status indicator; **Start / Stop / Restart** buttons (enabled per state); reachable URL `https://<lan-ip>:<port>`; all LAN IPs other devices can use; live request counter; a clear one-line explanation of what "running" means and how other devices connect. **No _ in any user-facing label** (§6.L 60th lesson).

3.4 Storage & TLS

- SQLite DB at `filesDir/lava-api.db`.
- **TLS**: generate a self-signed cert + key at first boot, persist under `filesDir`; serve HTTPS on the configured port (default 8443) so the on-device server matches the host's HTTPS surface. Cert-trust strategy for clients documented in sub-project 2/3.
- Bind `0.0.0.0` so other LAN devices reach it.

4. Network discovery identity (truthful + non-breaking)

- The on-device server **is** the Go engine on Android with SQLite, so it honestly advertises:
 - Service type `_lava-api._tcp` (release) / `_lava-api-dev._tcp` (debug) — same types the client already watches.
 - TXT: `engine=go, platform=android, storage=sqlite, version=<n>`.
- The existing **host** instance keeps advertising `engine=go, platform=server` (or absent platform) — **unchanged**; both appear in the client's list.
- The "distinct identity" the operator chose is realized via the **platform** TXT key: the client (sub-project 2) labels `platform=android` instances distinctly (e.g. "On this network · Android device") without any discovery re-architecture. Backward compatible: instances without `platform` render exactly as today.

5. Notification

- Dedicated channel via `core:notifications`.
- Persistent foreground notification while running: title "Lava API running"; body with reachable `https://<ip>:<port>` + live request count; **action buttons: Stop, Restart**; tap opens the landing screen.
- Stopped/Error states reflected (or notification removed on full stop).

6. Testing — maximal coverage, real evidence (§6.J / §6.Z / §6.AE)

Layer	What	Primary assertion
Go unit	config selector, sqlite storage impl	correct branch chosen; CRUD correctness
Go contract/parity	sqlite vs postgres identical JSON	field-level response equality
Go mobile	Start/Stop/Status real loopback HTTP	real JSON body on the wire
Android unit	ApiControlViewModel w/ real engine wrapper (fake only Service boundary); NSD TXT parsing	rendered state == server state
Instrumented Challenge	ChallengeNN_ApiAppBootAndServe: launch → Start → UI shows Running AND a real HTTPS request to the on-device server returns real JSON	user-visible UI text + on-the-wire JSON
Challenge	cold-start (C00-equiv), Stop, Restart, notification actions	user-visible state transitions

- Every Challenge carries the mandatory **falsifiability rehearsal** block (deliberately-broken-but-non-crashing mutation + observed assertion failure).
- Execution on the **host-direct** + **HVF** runner (per the §6.L 67th cycle resolution of the §6.X darwin/arm64 sub-debt) against a provisioned AVD; evidence under `.lava-ci-evidence/`.
- **§6.Z**: no APK of this app is distributed unless its Challenge set executed **green against the exact artifact**, with the evidence file present (matching commit SHA, ≤24h, BUILD SUCCESSFUL captured verbatim).

7. Risks & honest blockers

- **gomobile toolchain** must be present locally (Go 1.26 + gomobile/gobind + Android NDK). If absent, the .aar build is a hard blocker surfaced to the operator (no bluff).
- **On-device Challenge execution** needs a provisioned AVD reachable by the HVF runner; if absent, execution is honestly BLOCKED, not faked.
- **Firebase distribution** (sub-project 4) needs the new app's google-services.json, the shared keystore, and LAVA_FIREBASE_TOKEN; missing any → blocker, not bluff.
- **SQLite migration parity** with Postgres dialect is the main correctness risk; the parity test is the guard.

8. Roadmap for sub-projects 2–4 (committed sequencing)

1. **Client integration:** extend the existing onboarding API step + add Settings entry "Run API on this device": PackageManager check → if digital.vasic.lava.api[.dev] installed, launch it + trigger boot (it then appears in the instances list via mDNS); else deep-link to its Firebase/Play download. Clear copy eliminating confusion about controlling the local instance.
2. **Docs:** new docs/ON_DEVICE_API.md + diagrams (architecture, lifecycle, mDNS sequence); extend docs/ARCHITECTURE.md, docs/LOCAL_NETWORK_DISCOVERY.md, AGENTS.md, README.md, docs/CONTINUATION.md; user guide/manual.
3. **Distribution:** §6.Y version bumps; build all 3 apps both variants inside the container/host path; run full tests + Challenge matrix; §6.Z evidence; two-stage (§6.AA) debug→verify→release Firebase distribution via Firebase CLI, same keystore; commit+push all submodules + parent to all upstreams.

9. Definition of done (sub-project 1)

- lava-api-go runs on SQLite via config with all existing Postgres tests green + new parity tests green.
- :api-app builds (debug+release), boots the in-process Go server, serves real JSON over HTTPS on the LAN, advertises via mDNS with platform=android.

- Start/Stop/Restart/Status work from the landing UI; foreground notification with actions works.
- Full test matrix green with real evidence; Challenge set executed green against the built artifact (or honestly BLOCKED with the blocker named).